



ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA

D
BACH KHOA

TRÍ TUỆ NHÂN TẠO

N
A
N
G



Khoa Công Nghệ Thông Tin

TS. Nguyễn Văn Hiệu



ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA

Khoa Công Nghệ Thông Tin
TS. Nguyễn Văn Hiệu



TRÍ TUỆ NHÂN TẠO

Neural Networks

D
BACH KHOA

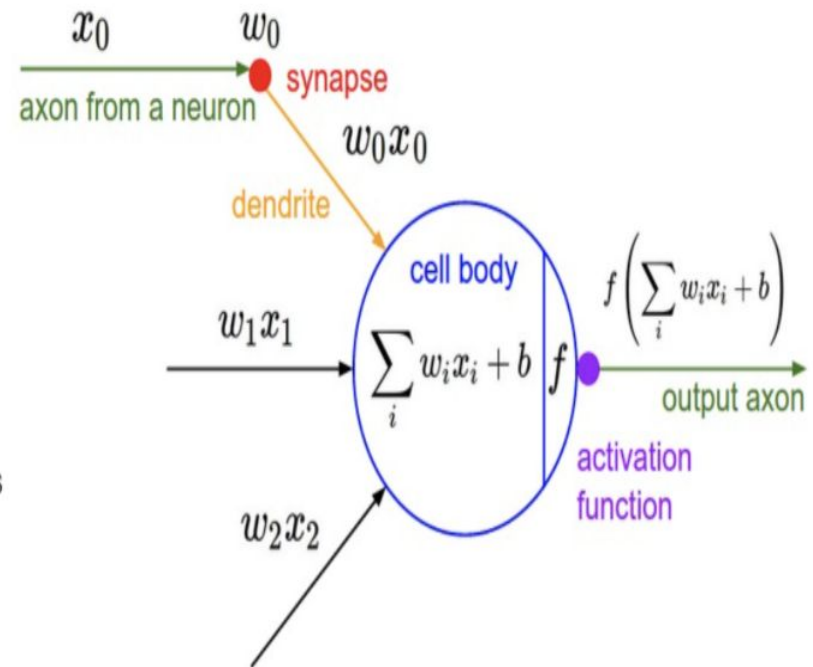
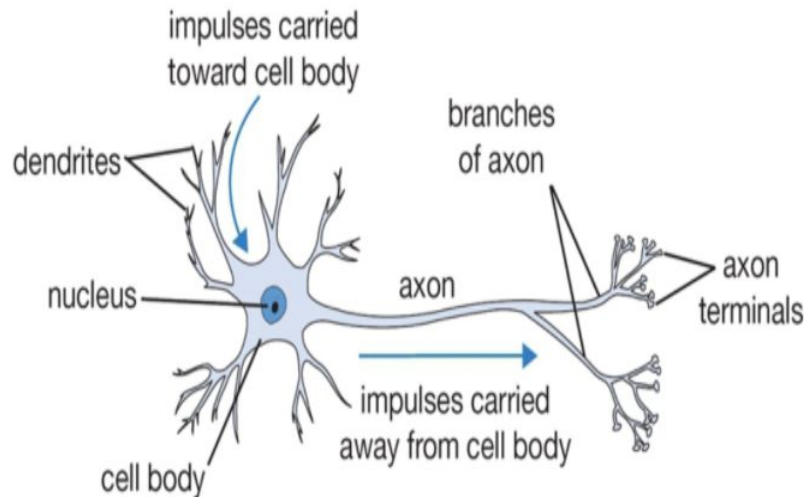
N
A
N
G

Nội dung

- Giới thiệu
- Neural Networks
- Demo

Giới thiệu

- Mạng nơ-ron được cấu thành từ các nơron đơn lẻ được gọi Perceptron
- Sigmoid Neurons



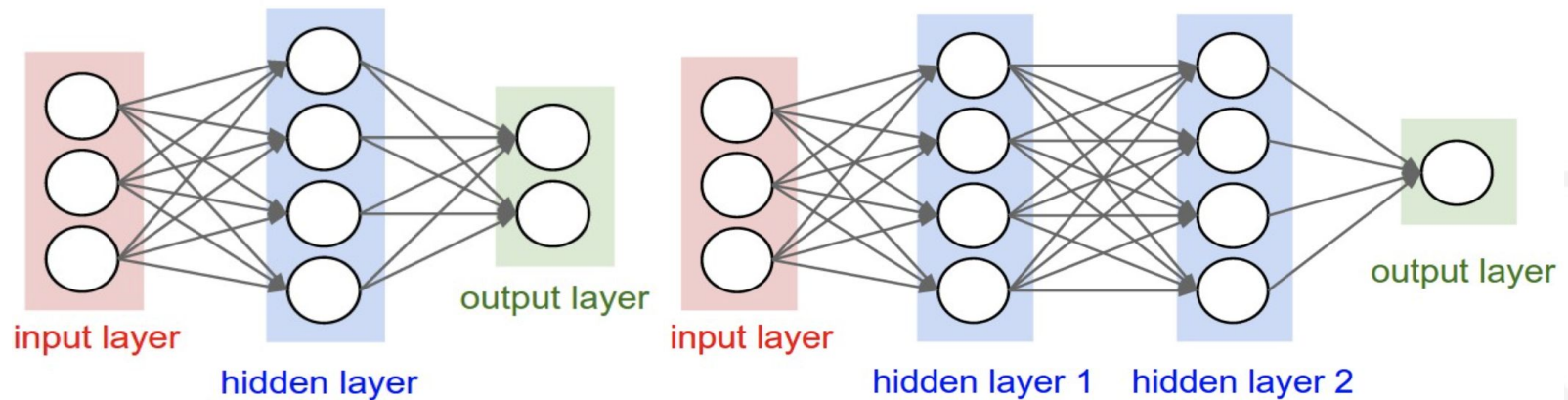
Giới thiệu

- Sigmoid Neurons

```
class Neuron(object):  
    # ...  
    def forward(self, inputs):  
        # assume inputs and weights and bias  
        cell_body_sum = np.sum(inputs * self.weights) + self.bias  
        # sigmoid activation function  
        firing_rate = 1.0 / (1.0 + math.exp(-cell_body_sum))  
    return firing_rate
```

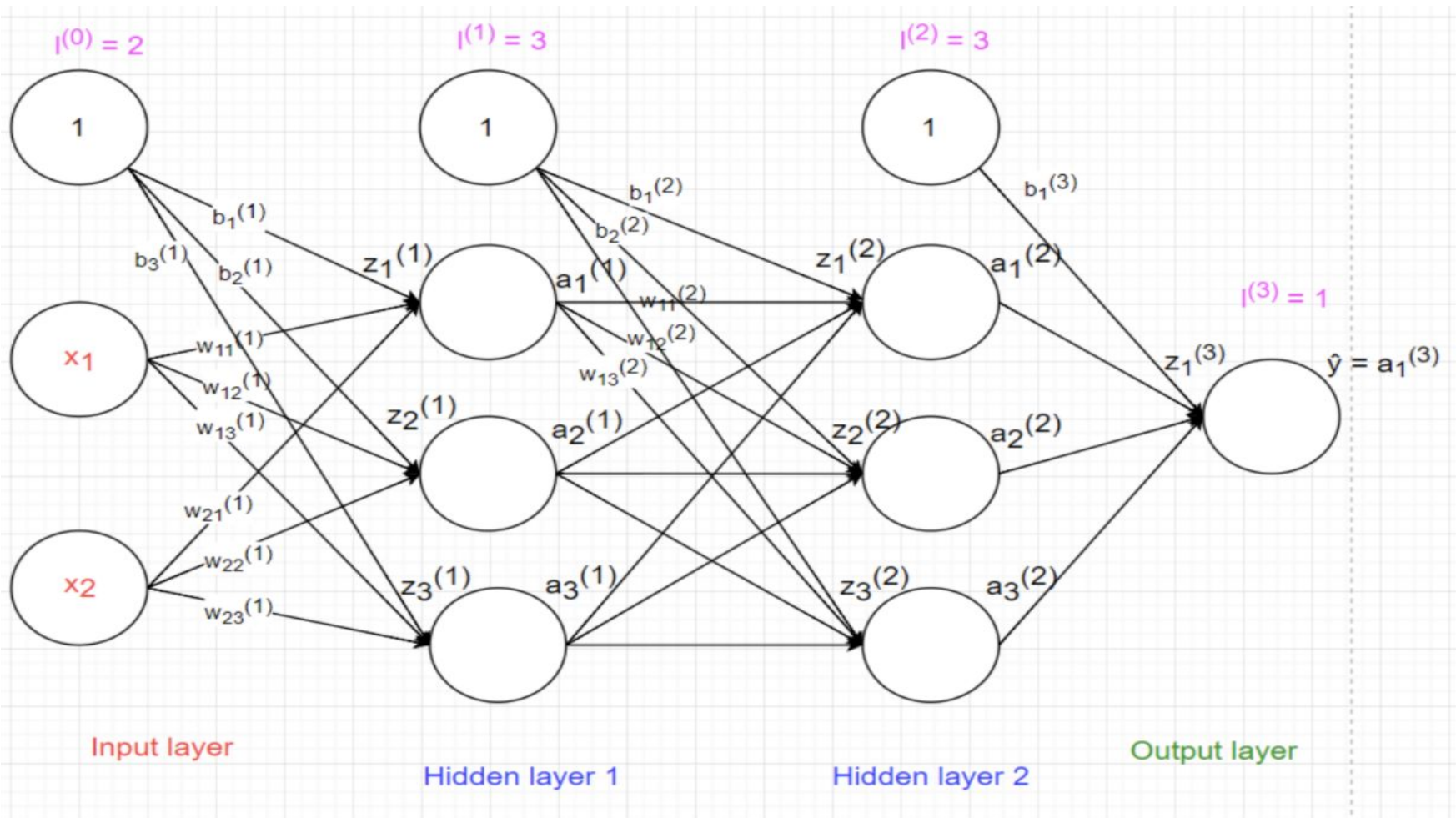
Kiến trúc Neural Networks

- Mạng Neural networks có 3 kiểu tầng



- Mỗi node trong hidden layer và output layer:
 - Liên kết tất cả với các node ở layer trước đó với hệ số riêng w
 - Mỗi node có một hệ số bias riêng
 - Diễn ra 2 bước: tính tổng linear và tính activation function

Kiến trúc Neural Networks

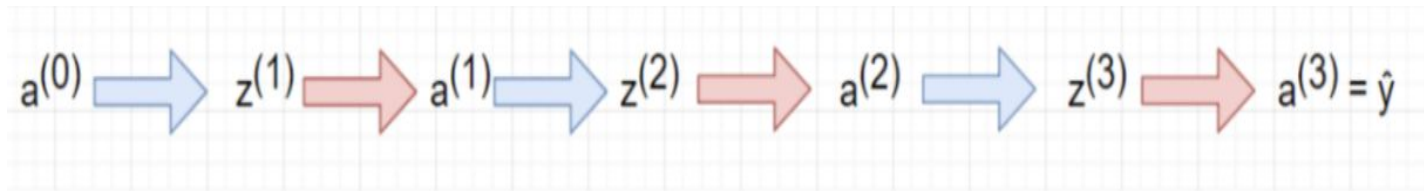


Kiến trúc Neural Networks

- $W^{(1)} \in R^{2 \times 3}, W^{(2)} \in R^{3 \times 3}, W^{(3)} \in R^{3 \times 1}$
- $b^{(1)} \in R^{3 \times 1}, b^{(2)} \in R^{3 \times 1}, b^{(3)} \in R$
- $z^{(1)} \in R^{3 \times 1}, z^{(2)} \in R^{3 \times 1}, z^{(3)} \in R$
- $a^{(0)} = x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, a^{(1)} \in R^{3 \times 1}, a^{(2)} \in R^{3 \times 1}, a^{(3)} \in R$

Lan truyền tiến

•



$$a^{(0)} = [x] = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$z_j^{(l+1)} = \sum_{i=1}^{n(l)} w_{ij}^{(l+1)} a_i^{(l)} + b_j^{(l+1)}$$

$$a_j^{(l+1)} = \delta(z_j^{(l+1)})$$

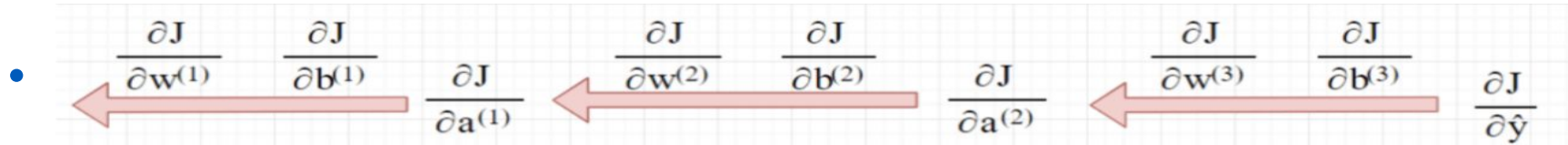
Học với Neural Network

- Loss function

$$J = -\frac{1}{m} \cdot \sum_{i=1}^m \sum_{k=1}^K \left(y_k^{(i)} \log \overbrace{\hat{y}_k^{(i)}}^{a_k^L} + (1 - y_k^{(i)}) \log (1 - \hat{y}_k^{(i)}) \right)$$

- SGD: $\Phi_w J$, $\Phi_b J$

Lan truyền ngược



- Bước 1:

$$\frac{dJ}{dy} = \frac{dJ}{da^{(3)}}$$

- Bước 2:

$$\frac{dJ}{dw^{(3)}} = \frac{dJ}{dy} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot \frac{dz^{(3)}}{dw^{(3)}} = \frac{dJ}{dy} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot a^{(2)}$$

$$\frac{dJ}{db^{(3)}} = \frac{dJ}{dy} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot \frac{dz^{(3)}}{db^{(3)}} = \frac{dJ}{dy} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot (1)$$

$$\frac{dJ}{da^{(2)}} = \frac{dJ}{dy} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot \frac{dz^{(3)}}{da^{(2)}} = \frac{dJ}{dy} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot \left(w^{(3)}\right)^T$$

Lan truyền ngược

- Bước 3:

$$\begin{aligned}\frac{dJ}{dw^{(2)}} &= \frac{dJ}{da^{(2)}} \cdot \frac{da^{(2)}}{dz^{(2)}} \cdot \frac{dz^{(2)}}{dw^{(2)}} = \frac{dJ}{da^{(2)}} \cdot \frac{da^{(2)}}{dz^{(2)}} \cdot a^{(1)} \\ \frac{dJ}{db^{(2)}} &= \frac{dJ}{da^{(2)}} \cdot \frac{da^{(2)}}{dz^{(2)}} \cdot \frac{dz^{(2)}}{db^{(2)}} = \frac{dJ}{da^{(2)}} \cdot \frac{da^{(2)}}{dz^{(2)}} \cdot (1) \\ \frac{dJ}{da^{(1)}} &= \frac{dJ}{da^{(2)}} \cdot \frac{da^{(2)}}{dz^{(2)}} \cdot \frac{dz^{(2)}}{da^{(1)}} = \frac{dJ}{da^{(2)}} \cdot \frac{da^{(2)}}{dz^{(2)}} \cdot \left(w^{(2)}\right)^T\end{aligned}$$

- Bước 4:

$$\begin{aligned}\frac{dJ}{dw^{(1)}} &= \frac{dJ}{da^{(1)}} \cdot \frac{da^{(1)}}{dz^{(1)}} \cdot a^{(0)} \\ \frac{dJ}{db^{(1)}} &= \frac{dJ}{da^{(1)}} \cdot \frac{da^{(1)}}{dz^{(1)}} \cdot (1)\end{aligned}$$

Demo

```
class NN():  
    def __init__(self, layers):  
        self.layers = layers  
        self.L = len(layers)  
        self.w = [np.random.randn(l2, l1 + 1)  
                   for l2, l1 in zip(layers[1:], layers[:-1])]
```

Demo

```
class NN():  
    def feedforward(self, x):  
        z = []  
        a = [self.add_bias(x)]  
        for l in range(1, self.L):  
            z_l = np.dot(self.w[l], a[l-1])  
            a_l = self.sigmoid(z_l)  
            if l < self.L - 1:  
                a_l = self.add_bias(a_l)  
  
            z.append(z_l)  
            a.append(a_l)  
  
        return (z, a)
```

Demo

```
class NN():  
    def backprop(self, x, y):  
        w_grad = [np.zeros(W.shape) for W in self.w]  
        z, a = self.feedforward(x) # feedforward  
        dz = a[-1] - y # backward  
        for _l in range(1, self.L):  
            l = -_l # layer index  
            if l < -1:  
                da = self.sigmoid_grad(z[l])  
                dz = np.dot(self.w[l+1][:, 1:].transpose(), dz) * da  
            w_grad[l] = np.dot(dz, a[l-1].transpose()) # gradient  
        return w_grad  
    def add_bias(self, a):  
        return np.insert(a, 0, 1, axis=0)  
    def sigmoid(self, z):  
        return 1.0 / (1.0 + np.exp(-z))  
    def sigmoid_grad(self, z):  
        s = self.sigmoid(z)  
        return s * (1 - s)
```



ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA

Khoa Công Nghệ Thông Tin
TS. Nguyễn Văn Hiệu



D
BACH KHOA

N
A
N
G